

UI and user experience guidelines for ads

Article • 01/04/2023

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

This article provides guidelines for providing great experiences with banner ads, interstitial ads, and native ads in your apps. For general guidance about how to design the look and feel for apps, see [Design & UI](#).

Important

Any use of advertising in your app must comply with the Microsoft Store Policies – including, without limitation, [policy 10.10](#) (Advertising Conduct and Content). In particular, your app's implementation of banner ads or interstitial ads must meet the requirements in Microsoft Store [policy 10.10.1](#). This article includes examples of implementations that would violate this policy. These examples are provided for informational purposes only, as a way to help you better understand the policy. These examples are not comprehensive, and there may be many other ways to violate the Microsoft Store Policies that are not listed in this article.

General best practices

Before reviewing our guidelines for different types of ads in this article, first review these general best practices to improve your ad revenue.

- [Plan your ad placements carefully](#) . See our related guidance about [optimizing the viewability of your ad units](#).
- [Use interstitial banner ads as a fallback for interstitial video ads](#) .
- [Know your users to serve better targeted ads](#) .
- [Use the latest advertising libraries](#) .
- [Set the correct COPPA settings for your app](#) .

Guidelines for banner ads

The following sections provide recommendations for how to implement [banner ads](#) in your app using [AdControl](#) and examples of implementations that violate [policy 10.10.1](#) of the Microsoft Store Policies.

Best practices

We recommend that you follow these best practices when you implement banner ads in your app:

- [Use Interactive Advertising Bureau sizes](#) [↗](#) that fit well with the layout for the device.
- Devote most of your app's UI to functional controls and content.
- Design advertising into your experience. Give your designers a sample ad to plan how the advertising is going to look. Two examples of well-planned ads in apps are the ads-as-content layout and the split layout.

To see how different ad sizes will look and function within your app during the development and testing phase, you can utilize our [test ad units](#). When you're done with testing, remember to [update your app with live ad units](#) before submitting the app for certification.

- Plan for times when no ads are available. There may be times when ads aren't being sent to your app. Lay out your pages in such a way that they look great whether they showcase an ad or not. For more information, see [Handle ad errors](#).
- If you have a scenario for alerting the user that is best handled with an overlay, call [AdControl.Suspend](#) while displaying the overlay and then call [AdControl.Resume](#) when the alert scenario is finished.

Practices to avoid

We recommend that you avoid these practices when you implement banner ads in your app:

- Don't bolt advertising into open real estate. Ad space shouldn't be placed into the first open piece of real estate you can find. Instead, it should be incorporated into your app's overall design.
- Don't over-advertise and saturate your app. Too many ads in your app detract from its appearance and usability. You want to make money with advertising, but not at the expense of the app itself.

- Don't distract user from their core tasks. The primary focus should always be on the app. The ad space should be incorporated so it remains a secondary focus.

Examples of policy violations

This section provides examples of banner ad scenarios that violate [policy 10.10.1](#) of the Microsoft Store Policies. These examples are provided for instructional purposes only, as a way to help you better understand the policy. These examples are not comprehensive, and there may be many other ways to violate policy 10.10.1 that are not listed here.

- Doing anything to interfere with the user's ability to view the banner ad, such as changing the opacity of the [AdControl](#) or placing another control on top of the **AdControl** (without first calling [AdControl.Suspend](#)).
- Requiring users to click on a banner ad to accomplish a task in your app, or forcing users to click on banner ads as a result of the design of your app.
- Bypassing the built-in minimum refresh timer for banner ads by any means, including (but not limited to) swapping [AdControl](#) objects or forcing a page refresh without user interaction.
- Using live ad units (that is, ad units that you obtain from Partner Center) during development and testing, or in an emulator.
- Writing or distributing code that calls ad services through means other than the Microsoft advertising libraries running in the context of your app.
- Interacting with undocumented interfaces or child objects created by the Microsoft advertising libraries, such as **WebView** or **MediaElement**.
- Placing ads in a viewbox to reduce the size of the ads in order to allow more ads on a page than normal.

Guidelines for interstitial ads

When used elegantly, [interstitial ads](#) can vastly increase your app revenue, without negatively impacting user satisfaction. When used improperly, such ads can have the exact opposite effect.

The following sections provide recommendations for how to implement interstitial video ads and interstitial banner ads in your app using [InterstitialAd](#), and examples of implementations that violate [policy 10.10.1](#) of the Microsoft Store Policies. Since you know your app better than anyone, except where policy is concerned, we leave it up to

you to make the best final decision. What's most important to keep in mind is that your app ratings and revenue are tightly coupled.

Best practices

We recommend that you follow these best practices when you implement interstitial ads in your app:

- Fit interstitial ads within the natural flow of the app, such as between game levels.
- Associate ads with tangible upsides, such as:
 - Hints towards level completion.
 - Extra time to retry a level.
 - Custom avatar features, like a tattoo or hat.
- If your app requires that an interstitial video ad be watched to completion, mention that rule upfront so they aren't surprised with an error message upon hitting the close button.
- Pre-fetch the ad (by calling [InterstitialAd.RequestAd](#)) ideally 30-60 seconds before you need to show it.
- Subscribe to all four events exposed in the [InterstitialAd](#) class (**Canceled**, **Completed**, **AdReady**, and **ErrorOccurred**) and use them to make the right decisions for your app.
- Have some built-in experience to use in lieu of a server-matched ad. You'll find this useful in a few scenarios:
 - Offline mode, when ad servers can't be reached.
 - When the **ErrorOccurred** event fires.
 - If you opt to save user bandwidth based on [ConnectionProfile](#), there are APIs in the **ConnectionProfile** class which can help.
- Use the default (30 second) timeout unless you have a valid reason to do otherwise, in which case don't go below 10 seconds. Interstitial ads take substantially longer to download than standard banner ads, especially in markets that don't have high speed connections.
- Be mindful of the user's data plan. For example, either don't show, or warn user, before serving an interstitial video ad on a mobile device that is near/over its data

limit. There are APIs in the [ConnectionProfile](#) class which can help.

- Continuously improve your app after the initial submission. Look at the [ad reports](#) and make design changes to improve fill and interstitial video completion rates.

Practices to avoid

We recommend that you avoid these practices when you implement interstitial ads in your app:

- Don't overdo it. Don't force ads more than every 5 minutes or so, unless the user explicitly engages with an optional tangible benefit, beyond just playing the game.
- Don't show interstitials at app launch, since users may believe they clicked the wrong tile.
- Don't show interstitials at exit. This is bad inventory, since completion rates will be near zero.
- Don't show two or more interstitial ads back to back. Users will be frustrated to see the ad progress bar reset to the starting point. Many will think it's just a coding or ad serving bug.
- Don't fetch an interstitial video ad more than 5 minutes before calling [InterstitialAd.Show](#). Good inventory will maximize the conversion of pre-fetched ads to billable impressions.
- Don't penalize a user for failures in ad serving, such as no ad available. For example, if you show a UI option to "Watch an ad to get xxx", you should provide xxx if the user did her part. Two options to consider:
 - Don't include the option unless the [InterstitialAd.AdReady](#) event has fired.
 - Have the app include a built-in experience that yields the same benefit as a real ad.
- Don't use interstitial ads to let a user gain a competitive advantage in a multi-player game. For example, don't entice the user with a better gun in a first-person shooter game if they view an interstitial ad. A custom shirt on the player's avatar is fine, so long as it doesn't provide camouflage!

Examples of policy violations

This section provides examples of interstitial ad scenarios that violate [policy 10.10.1](#) of the Microsoft Store Policies. These examples are provided for instructional purposes only, as a way to help you better understand the policy. These examples are not comprehensive, and there may be many other ways to violate policy 10.10.1 that are not listed here.

- Placing a UI element over the interstitial ad container.
- Calling [InterstitialAd.Show](#) while the user is engaged with the app.
- Using interstitial ads to obtain anything that may be consumed as a currency or traded with other users.
- Requesting a new interstitial ad in the context of the event handler for the [InterstitialAd.ErrorOccurred](#) event. This can result in an infinite loop and can cause operational issues for the advertising service.
- Requesting an interstitial ad merely to have a backup ad for a waterfall sequence of ads. If you request an interstitial ad and then receive the [InterstitialAd.AdReady](#) event, the next interstitial ad shown in your app must be the ad that is ready to be shown via the [InterstitialAd.Show](#) method.
- Using live ad units (that is, ad units that you obtain from Partner Center) during development and testing, or in an emulator.
- Writing or distributing code that calls ad services through means other than the Microsoft advertising libraries running in the context of your app.
- Interacting with undocumented interfaces or child objects created by the Microsoft advertising libraries, such as **WebView** or **MediaElement**.

Guidelines for native ads

[Native ads](#) give you have a lot of control over how you present advertising content to your users. Follow these requirements and guidelines to help ensure that the advertiser's message reaches your users while also helping to avoid delivering a confusing native ads experience to your users.

Register the container for your native ad

In your code, you must call the [RegisterAdContainer](#) method of the [NativeAdV2](#) object to register the UI element that acts as a container for the native ad and optionally any

specific controls that you want to register as clickable targets for the ad. This is required to properly track ad impressions and clicks.

There are two overloads for the **RegisterAdContainer** method that you can use:

- If you want the entire container for all the individual native ad elements to be clickable, call the **RegisterAdContainer(FrameworkElement)** method and pass the container control to the method. For example, if you display all of the native ad elements in separate controls that are all hosted in a **StackPanel** and you want the entire **StackPanel** to be clickable, pass the **StackPanel** to this method.
- If you want only certain native ad elements to be clickable, call the **RegisterAdContainer(FrameworkElement, IVector(FrameworkElement))** method. Only the controls that you pass to the second parameter will be clickable.

Required native ad elements

At a minimum, you must always show the following native ad elements provided by properties of the [NativeAdV2](#) object to the user in your native ad design. If you fail to include these elements, you may see poor performance and low yields for your ad unit.

1. Always display the title of the native ad (available in the **Title** property). Provide enough space to display at least 25 characters. If the title is longer, replace the additional text with an ellipsis.
2. Always display least one of the following elements to help differentiate the native ad experience from the rest of your app and clearly call out that the content is provided by an advertiser:
 - The distinguishable *ad* icon (available in the **AdIcon** property). This icon is supplied by Microsoft.
 - The *sponsored by* text (available in the **SponsoredBy** property). This text is supplied by the advertiser.
 - As an alternative to the *sponsored by* text, you can choose to display some other text that helps differentiate the native ad experience from the rest of your app, such as "Sponsored content", "Promotional content", "Recommended content", etc.

User experience

Your native ad should be clearly delineated from the rest of your app and have space around it to prevent accidental clicks. Use borders, different backgrounds, or some other UI to separate the ad content from the rest of your app. Keep in mind that accidental ad

clicks are not beneficial for your ad-based revenue or your end user experience in the long term.

Description

If you choose to show the description for the ad (available in the **Description** property of the **NativeAdV2** object), provide enough space to display at least 75 characters. We recommend that you use an animation to show the full content of the ad description.

Call to action

The *call to action* text (available in the **CallToAction** property of the **NativeAdV2** object) is a critical component of the ad. If you choose to show this text, follow these guidelines:

- Always display the *call to action* text to the user on a clickable control such as a button or hyperlink.
- Always display the *call to action* text in its entirety.
- Ensure that the *call to action* text is separated from the rest of the promotional text from the advertiser.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Optimize the viewability of your ad units

Article • 11/01/2022

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

The [Advertising performance report](#) includes viewability metrics for your ad units. Viewability is an important metric because the advertising industry is moving towards valuing viewable impressions rather than just delivered impressions. Advertisers tend to bid for viewable impressions because they have an increased chance of their ads being seen by users.

In alignment with the IAB viewability guidelines, a banner ad impression is counted as viewable if it meets the following criteria:

- Pixel requirement: Greater than or equal to 50% of the pixels in the advertisement were on the viewable space of the app.
- Time requirement: The time the pixel requirement is met was greater than or equal to one continuous second, post ad render.

A video ad impression is counted as viewable if it meets the following criteria:

- Pixel requirement: Greater than or equal to 50% of the pixels in the advertisement were on the viewable part of the app.
- Time requirement: The video met the pixel requirement and played for two continuous seconds post ad render.

Viewability is calculated using the following formula:

Viewability = [Viewed impressions] * 100 / [Total ad impressions]

Guidelines to improve ad unit viewability

To optimize the viewable impressions for your ad units, follow these guidelines.

1. **Measure performance** Use the [Advertising performance report](#) to review the viewability metrics of each of your ad units.

2. **Position your ad control appropriately** In general, the most viewable position for an ad is just above the fold (that is, at the bottom of the visible portion of the page that users can see without having to scroll). Ads that are displayed at top of the page are not as viewable. Consider inspecting each element of your page to see how you can optimize the viewable space in your app. Make sure your ad controls aren't placed in the app background.
3. **Manage page length** Shorter page content results in higher viewability. Implement infinite scrolling if your app pages need to be longer.
4. **Fix the ad position** Make sure your ads stay at a fixed position even as the user scrolls. This will help you get a higher view time and increase your viewability rate.
5. **Set opacity** Ensure the opacity of the container that contains the ad control is 100%.
6. **Implement lazy loading** Don't load your ads until the users scroll into the view that contains the ad control. This will ensure the ad displays longer for the user to view.
7. **Experiment with various ad sizes** Select a few ad sizes and measure viewability for each of them using the [Advertising performance report](#). Pick the one that works best for you.
8. **Revisit app design** Make improvements to your app page design to reduce page load time. Users tend to stay longer on pages that load quicker.

Handle ad errors

Article • 03/16/2023

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

The [AdControl](#), [InterstitialAd](#), and [NativeAdsManagerV2](#) classes each have an **ErrorOccurred** event that is raised if an ad-related error occurs. Your app code can handle this event and examine the [ErrorCode](#) and [ErrorMessage](#) properties of the event args object to help determine the cause of the error.

XAML apps

To handle ad-related errors in a XAML app:

1. Assign the **ErrorOccurred** event of your **AdControl**, **InterstitialAd**, or **NativeAdsManagerV2** object to the name of an event handler delegate.
2. Code the error event handling delegate so that it takes two parameters: an **Object** for the sender and an [AdErrorEventArgs](#) object.

Here is an example that assigns a delegate named **OnAdError** to the **ErrorOccurred** event of an **AdControl** object named *myBannerAdControl*.

C#

```
myBannerAdControl.ErrorOccurred = OnAdError;
```

Here is an example definition of the **OnAdError** delegate that writes error information to the output window in Visual Studio.

C#

```
private void OnAdError(object sender, AdErrorEventArgs e)
{
    System.Diagnostics.Debug.WriteLine("AdControl error (" +
        ((AdControl)sender).Name + "): " + e.Error +
        " ErrorCode: " + e.ErrorCode.ToString());
}
```

See [Error handling in XAML/C# walkthrough](#) for a walkthrough that demonstrates **AdControl** error handling in XAML and C#.

JavaScript/HTML apps

To handle **ErrorOccur** errors in a JavaScript app:

1. Assign the **onErrorOccurred** event to an event handler.
2. Code the event handler.

Here is an example that assigns an event handler named **errorLogger** to the **ErrorOccurred** event of an **AdControl** object.

HTML

```
<div id="myAd" style="position: absolute; top: 53px; left: 0px; width: 250px; height: 250px; z-index: 1" data-win-control="MicrosoftNSJS.Advertising.AdControl" data-win-options="{applicationId: '00001111-aaaa-2222-bbbb-3333cccc4444', adUnitId: 'test', onErrorOccurred: errorLogger}">
</div>
```

The error handling function is declarative and must be enclosed in the [markSupportedForProcessing](#) function.

The error handler catches the JavaScript error object when an error occurs. The error object provides two arguments to the error handler.

Here is an example of an error handling function named **errorLogger** that handles the **onErrorOccurred** event.

JavaScript

```
WinJS.Utilities.markSupportedForProcessing(
window.errorLogger = function (sender, evt) {
    console.log(new Date()).toLocaleTimeString() + ": " + sender.element.id
+ " error: " + evt.errorMessage +
    " error code: " + evt.errorCode + "\n");
});
```

See [Error Handling in JavaScript walkthrough](#) for a walkthrough that demonstrates **AdControl** error handling in JavaScript.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Error handling in XAML/C# walkthrough

Article • 03/16/2023

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

This walkthrough demonstrates how to catch ad-related errors in your app. This walkthrough uses an [AdControl](#) to display a banner ad, but the general concepts in it also apply to interstitial ads and native ads.

These examples assume that you have a XAML/C# app that contains an **AdControl**. For step-by-step instructions that demonstrate how to add an **AdControl** to your app, see [AdControl in XAML and .NET](#).

1. In your MainPage.xaml file, locate the definition for the **AdControl**. That code looks like this.

XML

```
<UI:AdControl
  ApplicationId="00001111-aaaa-2222-bbbb-3333cccc4444"
  AdUnitId="test"
  HorizontalAlignment="Left"
  Height="250"
  Margin="10,10,0,0"
  VerticalAlignment="Top"
  Width="300" />
```

2. After the **Width** property, but before the closing tag, assign a name of an error event handler to the [ErrorOccurred](#) event. In this walkthrough, the name of the error event handler is **OnAdError**.

```
xml <UI:AdControl ApplicationId="00001111-
aaaa-2222-bbbb-3333cccc4444" AdUnitId="test" HorizontalAlignment="Left"
Height="250" Margin="10,10,0,0" VerticalAlignment="Top" Width="300"
ErrorOccurred="OnAdError"/>
```

3. To generate an error at runtime, create a second **AdControl** with a different application ID. Because all **AdControl** objects in an app must use the same application ID, creating an additional **AdControl** with a different application id will throw an error.

Define a second **AdControl** in MainPage.xaml just after the first **AdControl**, and set the **ApplicationId** property to zero ("0").

XML

```
<UI:AdControl
    ApplicationId="0"
    AdUnitId="test"
    HorizontalAlignment="Left"
    Height="250"
    Margin="10,265,0,0"
    VerticalAlignment="Top"
    Width="300"
    ErrorOccurred="OnAdError" />
```

4. In MainPage.xaml.cs, add the following **OnAdError** event handler to the **MainPage** class. This event handler writes information to the Visual Studio **Output** window.

C#

```
private void OnAdError(object sender, AdErrorEventArgs e)
{
    System.Diagnostics.Debug.WriteLine("AdControl error (" +
        ((AdControl)sender).Name +
        "): " + e.ErrorMessage + " ErrorCode: " +
        e.ErrorCode.ToString());
}
```

5. Build and run the project. After the app is running, you will see a message similar to the one below in the **Output** window of Visual Studio.

JSON

```
AdControl error (): MicrosoftAdvertising.Shared.AdException: all ad
requests must use the same application ID within a single application
(0, 11112222-bbbb-3333-cccc-4444dddd5555) ErrorCode:
ClientConfiguration
```

Related topics

- [Advertising samples on GitHub](#) ↗

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Error handling in JavaScript walkthrough

Article • 09/25/2024

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

This walkthrough demonstrates how to catch ad-related errors in your JavaScript app. This walkthrough uses an [AdControl](#) to display a banner ad, but the general concepts in it also apply to interstitial ads and native ads.

These examples assume that you have a JavaScript app that contains an **AdControl**. For step-by-step instructions that demonstrate how to add an **AdControl** to your app, see [AdControl in HTML 5 and JavaScript](#). For a complete sample project that demonstrates how to add banner ads to a JavaScript/HTML app, see the [advertising samples on GitHub](#) .

1. In the default.html file, add a value for the **onErrorOccurred** event where you define the **data-win-options** in the **div** for the **AdControl**. Find the following code in the default.html file.

HTML

```
<div id="myAd" style="position: absolute; top: 53px; left: 0px; width: 300px; height: 250px; z-index: 1" data-win-control="MicrosoftNSJS.Advertising.AdControl" data-win-options="{applicationId: '00001111-aaaa-2222-bbbb-3333cccc4444', adUnitId: 'test'}">
</div>
```

Following the **adUnitId** attribute, add the value for the **onErrorOccurred** event.

HTML

```
<div id="myAd" style="position: absolute; top: 53px; left: 0px; width: 300px; height: 250px; z-index: 1" data-win-control="MicrosoftNSJS.Advertising.AdControl" data-win-options="{applicationId: '00001111-aaaa-2222-bbbb-3333cccc4444', adUnitId: 'test', onErrorOccurred: errorLogger}">
```

...

2. Create a **div** that will display text so you can see the messages being generated. To do this, add the following code after the **div** for **myAd**.

HTML

```
<div style="position:absolute; width:100%; height:130px; top:300px; left:0px">
    <b>Ad Events</b><br />
    <div id="adEvents" style="width:100%; height:110px; overflow:auto">
</div>
</div>
```

3. Create an **AdControl** that will trigger an error event. There can only be one application id for all **AdControl** objects in an app. So creating an additional one with a different application id will trigger an error at runtime. To do this, after the previous **div** sections you have added, add the following code to the body of the default.html page.

HTML

```
<!-- Because only one applicationId can be used, the following ad
control will fire an error event. -->
<div id="liveAd" style="position: absolute; top:500px; left:0px;
width:480px; height:80px"
    data-win-control="MicrosoftNSJS.Advertising.AdControl"
    data-win-options="{applicationId: '00000000-0000-0000-0000-
000000000000', adUnitId: 'test', onErrorOccurred: errorLogger }" >
</div>
```

4. In the project's default.js file, after the default initialization function, you will add the event handler for **errorLogger**. Scroll to the end of the file and after the last semi-colon is where you will put the following code.

JavaScript

```
WinJS.Utilities.markSupportedForProcessing(
window.errorLogger = function (sender, evt) {
    adEvents.innerHTML = (new Date()).toLocaleTimeString() + ": " +
    sender.element.id + " error: " + evt.errorMessage + " error code: "
+
    evt.errorCode + "<br>" + adEvents.innerHTML;
    console.log("errorhandler hit. \n");
});
```

5. Build and run the file. You will see the original ad from the sample app you built previously and text under that ad describing the error. You will not see the ad with the id of **liveAd**.

Related topics

- [Advertising samples on GitHub](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Known issues and troubleshooting for ads in apps

Article • 01/04/2023

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

This topic lists the known issues with the current release of the Microsoft Advertising SDK. For additional troubleshooting guidance, see the following topics.

- [HTML and JavaScript troubleshooting guide](#)
- [XAML and C# troubleshooting guide](#)

AdControl interface unknown in XAML

The XAML markup for an [AdControl](#) may incorrectly show a blue curvy line implying that the interface is unknown. This occurs only when targeting x86, and it may be ignored.

lastError from previous ad request

If there is a leftover **lastError** from the previous ad request, the event may be fired twice during the next ad call. While the new ad request will still be made and may yield a valid ad, this behavior may cause confusion.

Interstitial ads and navigation buttons on phones

On phones (or emulators) that have software **Back**, **Start**, and **Search** buttons instead of hardware buttons, the countdown timer and click through buttons for interstitial ads may be obscured.

Recently created ads are not being served to your app

If you have created an ad recently (less than a day), it might not be available immediately. If the ad has been approved for editorial content, it will be served once the advertising server has processed it and the ad is available as inventory.

No ads are shown in your app

There are many reasons you may see no ads, including network errors. Other reasons might include:

- Selecting an ad unit in Partner Center with a size that is greater or less than the size of the **AdControl** in your app's code.
- Ads will not appear if you're using a [test mode value](#) for your ad unit ID when running a live app.
- If you created a new ad unit ID in the past half-hour, you might not see an ad until the servers propagate new data through the system. Existing IDs that have shown ads before should show ads immediately.

If you can see test ads in the app, your code is working and is able to display ads. If you encounter issues, contact [product support](#). On that page, choose **Contact us**.

You can also post a question in the [forum](#) .

Test ads are showing in your app instead of live ads

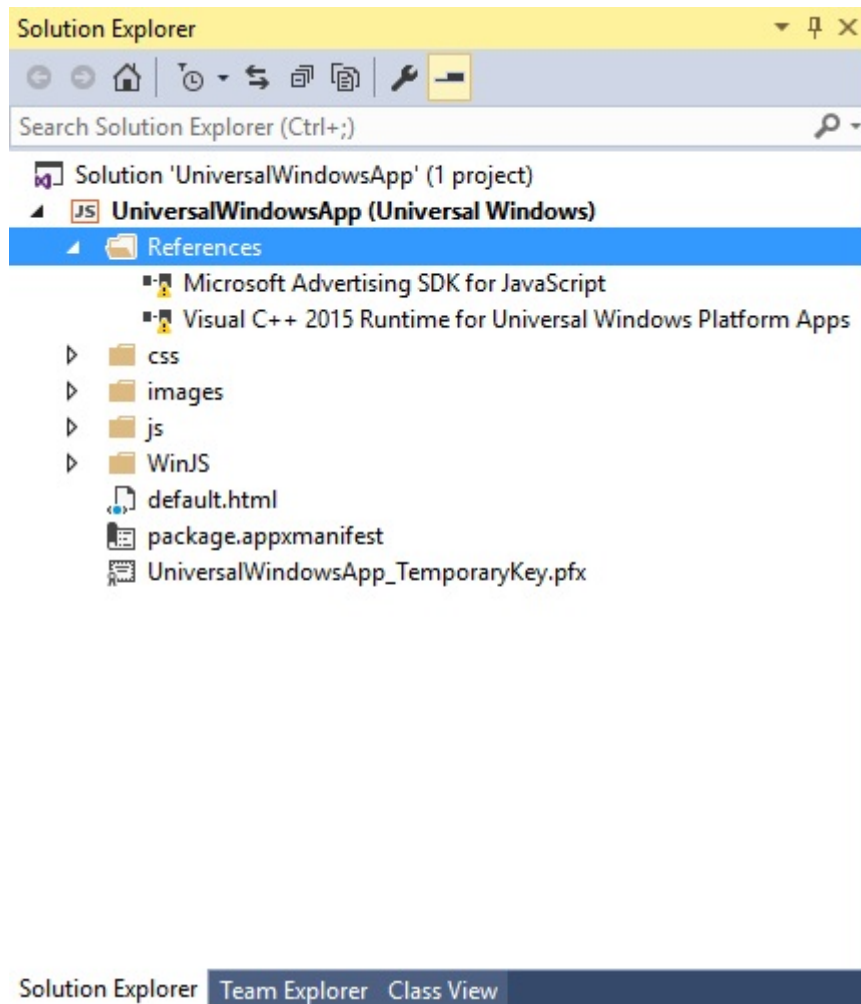
Test ads can be shown, even when you are expecting live ads. This can happen in the following scenarios:

- Our advertising platform cannot verify or find the live application ID used in the Store. In this case, when an ad unit is created by a user, its status can start as live (non-test) but will move to test status within 6 hours after the first ad request. It will change back to live if there are no requests from test apps for 10 days.
- Side-loaded apps or apps that are running in the emulator will not show live ads.

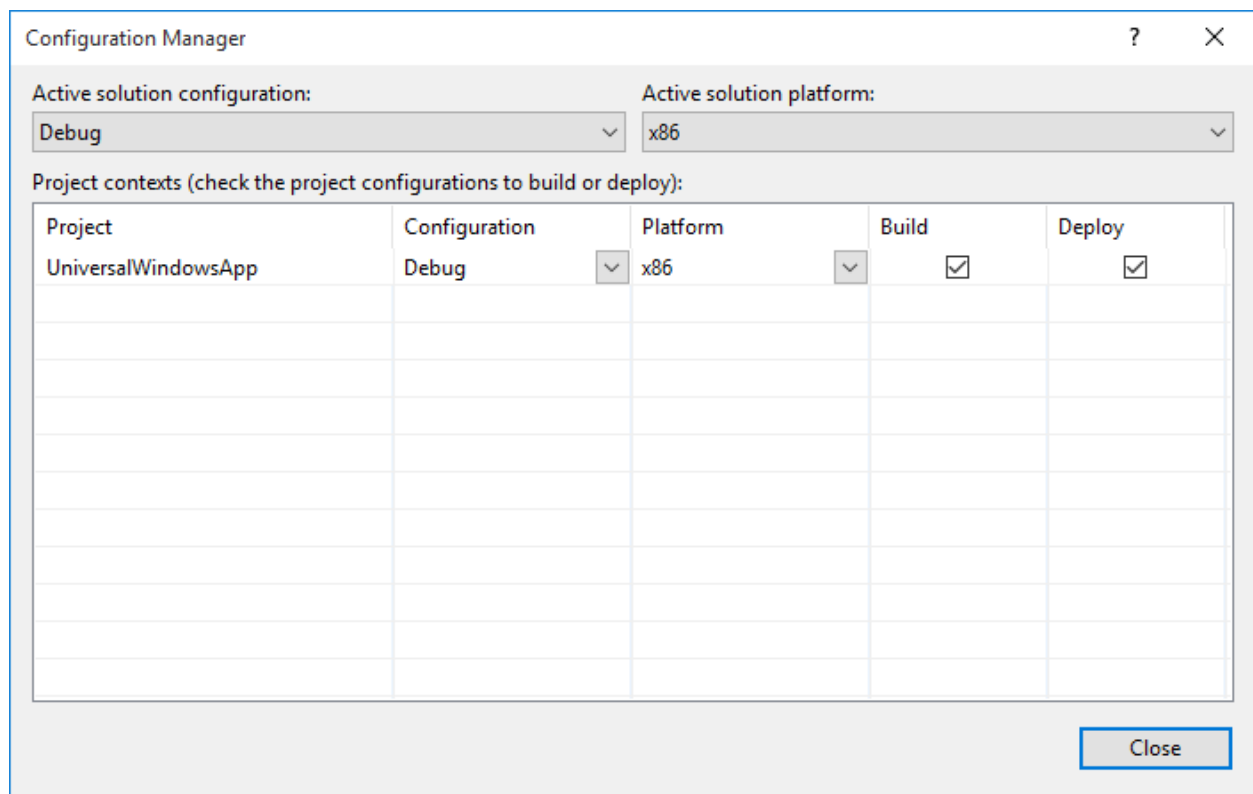
When a live ad unit is serving test ads, the ad unit's status shows **Active and serving test ads** in Partner Center. This does not currently apply to phone apps.

Reference errors caused by targeting Any CPU in your project

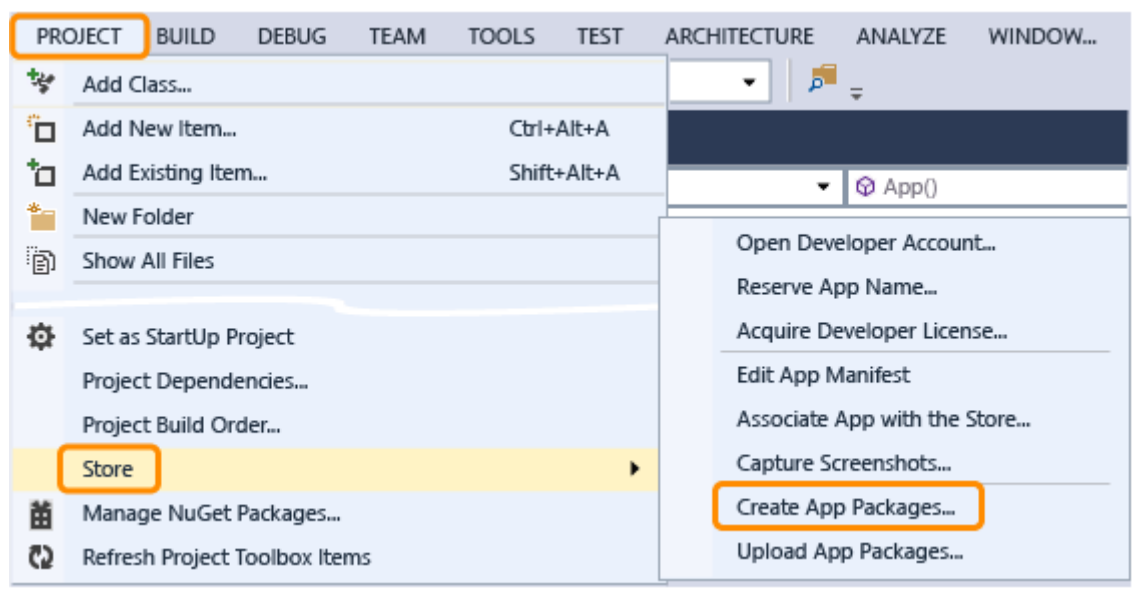
When using the Microsoft Advertising SDK, you cannot target **Any CPU** in your project. If your project targets the **Any CPU** platform, you may see a warning after adding the reference similar to this one.



To remove this warning, update your project to use an architecture-specific build output (for example, **x86**). Use **Configuration Manager** to set the platform targets for debug and release configurations.



When you create your app packages for store submission (as shown in the following images), be sure to include the architectures you intend to target. You may opt to skip x64 if you intend to run x86 builds on the x64 OS.



Create App Packages

 **Select and Configure Packages**

Output location:
 ...

Version:
 . . .
☒ Automatically increment

Generate app bundle:
 [What does an app bundle mean?](#)

Select the packages to create and the solution configuration mappings:

	Architecture	Solution Configuration
☐	Neutral	Release (Any CPU)
☒	x86	Release (x86)
☒	x64	Release (x64)
☒	ARM	Release (ARM)

☒ Include public symbol files, if any, to enable crash analysis for the app

Z-order in JavaScript/HTML apps

JavaScript/HTML apps must not place elements into the reserved MAX-10 range of z-order. The sole exception is an interrupt overlay, such as an inbound call notification for a Skype app.

Do not use borders

Setting border-related properties inherited by the **AdControl** from its parent class will cause the ad placement to be wrong.

More Information

For more information about the latest known issues and to post questions related to the Microsoft Advertising SDK, visit the [forum](#).

HTML and JavaScript troubleshooting guide

Article • 01/04/2023

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

This topic contains solutions to common development issues with the Microsoft advertising libraries in JavaScript/HTML apps.

- [HTML](#)
 - [AdControl not appearing](#)
 - [Black box blinks and disappears](#)
 - [Ads not refreshing](#)
- [JavaScript](#)
 - [AdControl not appearing](#)
 - [Black box blinks and disappears](#)
 - [Ads not refreshing](#)

HTML

AdControl not appearing

1. Ensure that the **Internet (Client)** capability is selected in Package.appxmanifest.
2. Ensure the JavaScript reference is present. Without the ad.js reference in the `<head>` section (after the default.js reference) the **AdControl** will be unable to display and an error will occur during build.

HTML

```
<head>
...
<script src="//Microsoft.Advertising.Javascript/ad.js"></script>
...
</head>
```

3. Check the application ID and ad unit ID. These IDs must match the application ID and ad unit ID that you obtained in Partner Center. For more information, see [Set up ad units in your app](#).

HTML

```
<div id="myAd" style="position: absolute; top: 50px; left: 0px;
                    width: 250px; height: 250px; z-index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl"
    data-win-options="{applicationId: 'ApplicationID',
                      adUnitId: 'AdUnitID'}">

</div>
```

4. Check the **height** and **width** properties. These must be set to one of the [supported ad sizes for banner ads](#).

HTML

```
<div id="myAd" style="position: absolute; top: 50px; left: 0px;
                    width: 250px; height: 250px; z-index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl"
    data-win-options="{applicationId: 'ApplicationID',
                      adUnitId: 'AdUnitID'}">

</div>
```

5. Check the element positioning. The [AdControl](#) must be inside the viewable area.
6. Check the **visibility** property. This property must not be set to collapsed or hidden. This property can be set inline (as shown below) or in an external style sheet.

HTML

```
<div id="myAd" style="visibility: visible; position: absolute; top:
1025px;
                    left: 500px; width: 250px; height: 250px; z-
index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl"
    data-win-options="{applicationId: 'ApplicationID',
                      adUnitId: 'AdUnitID'}">

</div>
```

7. Check the **position** property. The position property must be set to an appropriate value depending on the element's other properties (for example, margins in parent element and z-index). This property can be set inline (as shown below) or in an external style sheet.

HTML

```
<div id="myAd" style="visibility: visible; position: absolute; top:
1025px;
                left: 500px; width: 250px; height: 250px; z-
index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl"
    data-win-options="{applicationId: 'ApplicationID',
                    adUnitId: 'AdUnitID'}">
</div>
```

8. Check the **z-index** property. The **z-index** property must be set high enough so the **AdControl** always appears on top of other elements. This property can be set inline (as shown below) or in an external style sheet.

HTML

```
<div id="myAd" style="visibility: visible; position: absolute; top:
1025px;
                left: 500px; width: 250px; height: 250px; z-
index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl"
    data-win-options="{applicationId: 'ApplicationID',
                    adUnitId: 'AdUnitID'}">
</div>
```

9. Check external style sheets. If properties are set on the **AdControl** element through an external style sheet, ensure all of the above properties are correctly set.

HTML

```
<div id="myAd" style="visibility: visible; position: absolute; top:
1025px;
                left: 500px; width: 250px; height: 250px; z-
index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl"
    data-win-options="{applicationId: 'ApplicationID',
                    adUnitId: 'AdUnitID'}">
</div>
```

10. Check the parent of the **AdControl**. If the **AdControl** resides in a parent element, the parent must be active and visible.

HTML

```
<div style="position: absolute; width: 500px; height: 500px;">
    <div id="myAd" style="position: relative; top: 0px; left: 100px;
                        width: 250px; height: 250px; z-index: 1"
        data-win-control="MicrosoftNSJS.Advertising.AdControl"
        data-win-options="{applicationId: 'ApplicationID',
```

```
adUnitId: 'AdUnitID'}">
</div>
</div>
```

11. Ensure the **AdControl** is not hidden from the viewport. The **AdControl** must be visible for ads to display properly.
12. Live values for [ApplicationId](#) and [AdUnitId](#) should not be tested in the emulator. To ensure the **AdControl** is functioning as expected, use the [test values](#) for both **ApplicationId** and **AdUnitId**.

Black box blinks and disappears

1. Double-check all steps in the previous [AdControl not appearing](#) section.
2. Handle the **onErrorOccurred** event, and use the message that is passed to the event handler to determine whether an error occurred and what type of error was thrown. More details can be found in [Error handling in JavaScript walkthrough](#).

HTML

```
<div id="myAd" style="position: absolute; top: 0px; left: 0px;
width: 728px; height: 90px; z-index: 1"
data-win-control="MicrosoftNSJS.Advertising.AdControl"
data-win-options="{applicationId: 'ApplicationID',
adUnitId: 'AdUnitID',
onErrorOccurred: errorLogger}">
</div>
<div style="position: absolute; width: 100%; height: 130px; top: 300px;
left: 0px">
  <b>Ad Events</b><br />
  <div id="adEvents" style="width: 100%; height: 110px; overflow: auto">
</div>
</div>
```

The most common error that causes a black box is "No ad available." This error means there is no ad available to return from the request.

3. The **AdControl** is behaving normally. By default, the **AdControl** will collapse when it cannot display an ad. If other elements are children of the same parent they may move to fill the gap of the collapsed **AdControl** and expand when the next request is made.

Ads not refreshing

1. Check the **isAutoRefreshEnabled** property. By default, this optional property is set to true. When set to false, the **refresh** method must be used to retrieve another ad.

HTML

```
<div id="myAd" style="position: absolute; top: 0px; left: 0px;
                    width: 250px; height: 250px; z-index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl"
    data-win-options="{ applicationId: 'ApplicationID',
                        adUnitId: 'AdUnitID',
                        onErrorOccurred: errorLogger,
                        isAutoRefreshEnabled: true}">

</div>
```

2. Check calls to the **refresh** method. When using automatic refresh, **refresh** cannot be used to retrieve another ad. When using manual refresh, **refresh** should be called only after a minimum of 30 to 60 seconds depending on the device's current data connection.

This example demonstrates how to use the **refresh** method. The following HTML code shows an example of how to instantiate the **AdControl** with **isAutoRefreshEnabled** set to false.

HTML

```
<div id="myAd" style="position: absolute; top: 0px; left: 0px;
                    width: 250px; height: 250px; z-index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl"
    data-win-options="{ applicationId: 'ApplicationID',
                        adUnitId: 'AdUnitID',
                        onErrorOccurred: errorLogger,
                        isAutoRefreshEnabled: false}">

</div>
```

This example demonstrates how to use the **refresh** function.

JavaScript

```
args.setPromise(WinJS.UI.processAll()
    .then(function (args) {
        window.setInterval(function()
        {
            document.getElementById("myAd").winControl.refresh();
        }, 60000)
    })
);
```

3. The **AdControl** is behaving normally. Sometimes the same ad will appear more than once in a row giving the appearance that ads are not refreshing.

JavaScript

AdControl not appearing

1. Ensure that the **Internet (Client)** capability is selected in Package.appxmanifest.
2. Ensure the **AdControl** is instantiated. If the **AdControl** is not instantiated, it will not be available.

The following snippets show an example of instantiating the **AdControl**. This HTML code shows an example of setting up the UI for the **AdControl**

HTML

```
<div id="myAd" style="position: absolute; top: 0px; left: 0px;
        width: 250px; height: 250px; z-index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl">
</div>
```

The following JavaScript code shows an example of instantiating the **AdControl**

JavaScript

```
app.onactivated = function (args) {
    if (args.detail.kind === activation.ActivationKind.launch) {
        if (args.detail.previousExecutionState !==
            activation.ApplicationExecutionState.terminated)
        {
            var adDiv = document.getElementById("myAd");
            var myAdControl = new
MicrosoftNSJS.Advertising.AdControl(adDiv,
            {
                applicationId: "{ApplicationID}",
                adUnitId: "{AdUnitID}"
            });
            myAdControl.onErrorOccurred = myAdError;
        } else {
            ...
        }
    }
}
```

3. Check the parent element. The parent `<div>` must be correctly assigned, active, and visible.

JavaScript

```
var adDiv = document.getElementById("myAd");
var myAdControl = new MicrosoftNSJS.Advertising.AdControl(adDiv, {
    applicationId: "{ApplicationID}",
    adUnitId: "{AdUnitID}"
});
```

4. Check the application ID and ad unit ID. These IDs must match the application ID and ad unit ID that you obtained in Partner Center. For more information, see [Set up ad units in your app](#).

JavaScript

```
var myAdControl = new MicrosoftNSJS.Advertising.AdControl(adDiv, {
    applicationId: "{ApplicationID}",
    adUnitId: "{AdUnitID}"
});
```

5. Check the parent element of the **AdControl**. The parent must be active and visible.
6. Live values for **ApplicationId** and **AdUnitId** should not be tested in the emulator. To ensure the **AdControl** is functioning as expected, use the [test values](#) for both **ApplicationId** and **AdUnitId**.

Black box blinks and disappears

1. Double-check all steps in the [AdControl not appearing](#) section.
2. Handle the **onErrorOccurred** event, and use the message that is passed to the event handler to determine whether an error occurred and what type of error was thrown. More details can be found in [Error handling in JavaScript walkthrough](#).

This example demonstrates how to implement an error handler that reports error messages. This snippet of HTML code provides an example of how to set up the UI to display error messages.

HTML

```
<div style="position:absolute; width:100%; height:130px; top:300px">
  <b>Ad Events</b><br />
  <div id="adEvents" style="width:100%; height:110px; overflow:auto">
```

```
</div>
</div>
```

This example demonstrates how to instantiate the **AdControl**. This function would be inserted in the app.onactivated file.

JavaScript

```
var myAdControl = new MicrosoftNSJS.Advertising.AdControl(adDiv,
{
    applicationId: "{ApplicationID}",
    adUnitId: "{AdUnitID}"
});
myAdControl.onErrorOccurred = myAdError;
```

This example demonstrates how to report errors. This function would be inserted below the self-running function in the default.js file.

JavaScript

```
WinJS.Utilities.markSupportedForProcessing
(
    window.errorLogger = function (sender, evt)
    {
        adEvents.innerHTML = (new Date()).toLocaleTimeString() + ": " +
            sender.element.id + " error: " + evt.errorMessage + " error
code: " +
            evt.errorCode + "<br>" + adEvents.innerHTML;
    }
);
```

The most common error that causes a black box is "No ad available." This error means there is no ad available to return from the request.

3. The **AdControl** is behaving normally. Sometimes the same ad will appear more than once in a row giving the appearance that ads are not refreshing.

Ads not refreshing

1. Check whether the **IsAutoRefreshEnabled** property of your **AdControl** is set to false. By default, this optional property is set to **true**. When set to **false**, the **Refresh** method must be used to retrieve another ad.
2. Check calls to the **Refresh** method. When using automatic refresh (**IsAutoRefreshEnabled** is **true**), **Refresh** cannot be used to retrieve another ad. When using manual refresh (**IsAutoRefreshEnabled** is **false**), **Refresh** should be

called only after a minimum of 30 to 60 seconds depending on the device's current data connection.

This example demonstrates how to create the **div** for the **AdControl**.

HTML

```
<div id="myAd" style="position: absolute; top: 0px; left: 0px;
        width: 250px; height: 250px; z-index: 1"
    data-win-control="MicrosoftNSJS.Advertising.AdControl">
</div>
```

This example shows how to use the **Refresh** function

JavaScript

```
var myAdControl = new MicrosoftNSJS.Advertising.AdControl(adDiv,
{
    applicationId: "{ApplicationID}",
    adUnitId: "{AdUnitID}",
    isAutoRefreshEnabled: false
});
...
args.setPromise(WinJS.UI.processAll()
    .then(function (args) {
        window.setInterval(function()
        {
            document.getElementById("myAd").winControl.refresh();
        }, 60000)
    })
);
```

3. The **AdControl** is behaving normally. Sometimes the same ad will appear more than once in a row giving the appearance that ads are not refreshing.

XAML and C# troubleshooting guide

Article • 01/04/2023

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

This topic contains solutions to common development issues with the Microsoft advertising libraries in XAML apps.

- [XAML](#)
 - [AdControl not appearing](#)
 - [Black box blinks and disappears](#)
 - [Ads not refreshing](#)
- [C#](#)
 - [AdControl not appearing](#)
 - [Black box blinks and disappears](#)
 - [Ads not refreshing](#)

XAML

AdControl not appearing

1. Ensure that the **Internet (Client)** capability is selected in Package.appxmanifest.
2. Check the application ID and ad unit ID. These IDs must match the application ID and ad unit ID that you obtained in Partner Center. For more information, see [Set up ad units in your app](#).

XML

```
<UI:AdControl AdUnitId="{AdUnitID}" ApplicationId="{ApplicationID}"
              Width="728" Height="90" />
```

3. Check the **Height** and **Width** properties. These must be set to one of the [Supported ad sizes for banner ads](#).

XML

```
<UI:AdControl AdUnitId="{AdUnitID}"
              ApplicationId="{ApplicationID}"
              Width="728" Height="90" />
```

4. Check the element position. The [AdControl](#) must be inside the viewable area.
5. Check the **Visibility** property. The optional **Visibility** property must not be set to collapsed or hidden. This property can be set inline (as shown below) or in an external style sheet.

XML

```
<UI:AdControl AdUnitId="{AdUnitID}"
              ApplicationId="{ApplicationID}"
              Visibility="Visible"
              Width="728" Height="90" />
```

6. Check the parent of the **AdControl**. If the **AdControl** element resides in a parent element, the parent must be active and visible.

XML

```
<StackPanel>
    <UI:AdControl AdUnitId="{AdUnitID}"
                  ApplicationId="{ApplicationID}"
                  Width="728" Height="90" />
</StackPanel>
```

7. Ensure the **AdControl** is not hidden from the viewport. The **AdControl** must be visible for ads to display properly.
8. Live values for **ApplicationId** and **AdUnitId** should not be tested in the emulator. To ensure the **AdControl** is functioning as expected, use the [test values](#) for both **ApplicationId** and **AdUnitId**.

Black box blinks and disappears

1. Double-check all steps in the previous [AdControl not appearing](#) section.
2. Handle the **ErrorOccurred** event, and use the message that is passed to the event handler to determine whether an error occurred and what type of error was thrown. See [Error handling in XAML/C# walkthrough](#) for more information.

This example demonstrates an **ErrorOccurred** event handler. The first snippet is the XAML UI markup.

XML

```
<UI:AdControl AdUnitId="{AdUnitID}"
              ApplicationId="{ApplicationID}"
              Width="728" Height="90"
              ErrorOccurred="adControl_ErrorOccurred" />
<TextBlock x:Name="TextBlock1" TextWrapping="Wrap" Width="500"
          Height="250" />
```

This example demonstrates the corresponding C# code.

C#

```
private void adControl_ErrorOccurred(object sender,
    Microsoft.Advertising.WinRT.UI.AdErrorEventArgs e)
{
    TextBlock1.Text = e.ErrorMessage;
}
```

The most common error that causes a black box is "No ad available." This error means there is no ad available to return from the request.

3. The [AdControl](#) is behaving normally.

By default, the **AdControl** will collapse when it cannot display an ad. If other elements are children of the same parent they may move to fill the gap of the collapsed **AdControl** and expand when the next request is made.

Ads not refreshing

1. Check the [IsAutoRefreshEnabled](#) property. By default, this optional property is set to **True**. When set to **False**, the [Refresh](#) method must be used to retrieve another ad.

XML

```
<UI:AdControl AdUnitId="{AdUnitID}"
              ApplicationId="{ApplicationID}"
              Width="728" Height="90"
              IsAutoRefreshEnabled="True" />
```

2. Check calls to the **Refresh** method. When using automatic refresh, **Refresh** cannot be used to retrieve another ad. When using manual refresh, **Refresh** should be called only after a minimum of 30 to 60 seconds depending on the device's current data connection.

The following code snippets show an example of how to use the **Refresh** method. The first snippet is the XAML UI markup.

XML

```
<UI:AdControl x:Name="adControl1"
    AdUnitId="{AdUnit_ID}"
    ApplicationId="{ApplicationID}"
    Width="728" Height="90"
    IsAutoRefreshEnabled="False" />
```

This code snippet shows an example of the C# code behind the UI markup.

C#

```
public RefreshAds()
{
    var timer = new DispatcherTimer() { Interval =
    TimeSpan.FromSeconds(60) };
    timer.Tick += (s, e) => adControl1.Refresh();
    timer.Start();
}
```

3. The **AdControl** is behaving normally. Sometimes the same ad will appear more than once in a row giving the appearance that ads are not refreshing.

C#

AdControl not appearing

1. Ensure that the **Internet (Client)** capability is selected in Package.appxmanifest.
2. Ensure the **AdControl** is instantiated. If the **AdControl** is not instantiated it will not be available.

C#

```
using Microsoft.Advertising.WinRT.UI;

namespace AdControlExample
{
    public sealed partial class MainPage : Page
    {
        AdControl myAdControl;

        public MainPage()
```

```

    {
        this.InitializeComponent();

        myAdControl = new AdControl()
        {
            ApplicationId = "{ApplicationID}",
            AdUnitId = "{AdUnitID}",
            Height = 90,
            Width = 728
        };
    }
}

```

3. Check the application ID and ad unit ID. These IDs must match the application ID and ad unit ID that you obtained in Partner Center. For more information, see [Set up ad units in your app](#).

C#

```

adControl = new AdControl();
adControl.ApplicationId = "{ApplicationID}"; adControl.AdUnitId = "{AdUnitID}";
adControl.Height = 90;
adControl.Width = 728;

```

4. Check the **Height** and **Width** parameters. These must be set to one of the [supported ad sizes for banner ads](#).

C#

```

adControl = new AdControl();
adControl.ApplicationId = "{ApplicationID}";
adControl.AdUnitId = "{AdUnitID}";
adControl.Height = 90; adControl.Width = 728;

```

5. Ensure the **AdControl** is added to a parent element. To display, the **AdControl** must be added as a child to a parent control (for example, a **StackPanel** or **Grid**).

C#

```

ContentPanel.Children.Add(adControl);

```

6. Check the **Margin** parameter. The **AdControl** must be inside the viewable area.
7. Check the **Visibility** property. The optional **Visibility** property must be set to **Visible**.

C#

```
adControl = new AdControl();
adControl.ApplicationId = "{ApplicationID}";
adControl.AdUnitId = "{AdUnitID}";
adControl.Height = 90;
adControl.Width = 728;
adControl.Visibility = System.Windows.Visibility.Visible;
```

8. Check the parent of the **AdControl**. The parent must be active and visible.
9. Live values for **ApplicationId** and **AdUnitId** should not be tested in the emulator. To ensure the **AdControl** is functioning as expected, use the [test values](#) for both **ApplicationId** and **AdUnitId**.

Black box blinks and disappears

1. Double-check all steps in the [AdControl not appearing](#) section above.
2. Handle the **ErrorOccurred** event, and use the message that is passed to the event handler to determine whether an error occurred and what type of error was thrown. See [Error handling in XAML/C# walkthrough](#) for more information.

The following examples show the basic code needed to implement an error call. This XAML code defines a **TextBlock** that is used to display the error message.

XML

```
<TextBlock x:Name="TextBlock1" TextWrapping="Wrap" Width="500"
Height="250" />
```

This C# code retrieves the error message and displays it in the **TextBlock**.

C#

```
using Microsoft.Advertising.WinRT.UI;

namespace AdControlExample
{
    public partial class MainPage : Page
    {
        AdControl myAdControl;

        public MainPage()
        {
            this.InitializeComponent();
        }
    }
}
```

```

myAdControl = new AdControl();
myAdControl.ApplicationId = "{ApplicationID}";
myAdControl.AdUnitId = "{AdUnitID}";
myAdControl.Height = 90;
myAdControl.Width = 728;

myAdControl.ErrorOccurred += (s,e) =>
{
    TextBlock1.Text = e.Error.Message;
};
}
}
}

```

The most common error that causes a black box is "No ad available." This error means there is no ad available to return from the request.

3. **AdControl** is behaving normally. Sometimes the same ad will appear more than once in a row giving the appearance that ads are not refreshing.

Ads not refreshing

1. Check whether the **IsAutoRefreshEnabled** property of your **AdControl** is set to false. By default, this optional property is set to **true**. When set to **false**, the **Refresh** method must be used to retrieve another ad.
2. Check calls to the **Refresh** method. When using automatic refresh (**IsAutoRefreshEnabled** is **true**), **Refresh** cannot be used to retrieve another ad. When using manual refresh (**IsAutoRefreshEnabled** is **false**), **Refresh** should be called only after a minimum of 30 to 60 seconds depending on the device's current data connection.

The following example demonstrates how to call the **Refresh** method.

```

C#

AdControl myAdControl;

public MainPage()
{
    InitializeComponent();

    myAdControl = new AdControl();
    myAdControl.ApplicationId = "{ApplicationID}";
    myAdControl.AdUnitId = "{AdUnitID}";
    myAdControl.Height = 90;
    myAdControl.Width = 728;
    myAdControl.IsAutoRefreshEnabled = false;
}

```



```
ContentPanel.Children.Add(myAdControl);

var timer = new DispatcherTimer() { Interval =
    TimeSpan.FromSeconds(60) };
timer.Tick += (s, e) => myAdControl.Refresh();
timer.Start();
}
```

3. The **AdControl** is behaving normally. Sometimes the same ad will appear more than once in a row giving the appearance that ads are not refreshing.

Release notes for the advertising libraries

Article • 10/20/2022

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

This section provides release notes for the current release of the Microsoft advertising libraries. These libraries support XAML and JavaScript/HTML apps for Windows 10, Windows 8.1, Windows Phone 8.1 and Windows Phone 8.

Installation

The Microsoft advertising libraries are available as part of the [Microsoft Advertising SDK](#) . For more information about installing the SDK, see [Install the Microsoft Advertising SDK](#).

Uninstall previous versions

Before you install the latest Microsoft Advertising SDK, it is highly recommended that you uninstall all prior instances of the SDK. For more information, see [Install the Microsoft Advertising SDK](#).

Target architecture-specific build outputs

When using the Microsoft advertising libraries, you cannot target **Any CPU** in your project. If your project targets the **Any CPU** platform, you may see a warning in your project after you add a reference to the Microsoft advertising libraries. To remove this warning, update your project to use an architecture-specific build output (for example, **x86**). For more information, see [Known issues](#).

C++ Support

The Microsoft advertising libraries (which include the **AdControl** and **InterstitialAd** classes) support apps written in C++ and DirectX using Windows Runtime

Interoperability (*interop*). For more information and examples about programming using XAML and C++, see [Type System](#).

No toolbox control

In the current release of the Microsoft advertising libraries in the [Microsoft Advertising SDK](#), there is no toolbox control for dragging an **AdControl** or **InterstitialAd** to a design surface in your app. For instructions about adding these controls in your markup and code, see the [developer walkthroughs](#).

Latitude and longitude properties no longer available

The **AdControl** class no longer has **Latitude** and **Longitude** properties for UWP apps. Instead, code built into the ad control will detect and send these values to the ad servers on the app's behalf.

Windows Premium Ads Publishers Program

Article • 10/24/2022

Warning

As of June 1, 2020, the Microsoft Ad Monetization platform for Windows UWP apps will be shut down. [Learn more](#) 

As part of our continuous efforts to help Windows developers monetize their apps through ads, we offer the Windows Premium Ads Publishers Program. This is a curated collection of ad-enabled apps and games that are best in class in terms of usage, engagement and experience. These apps and games are viewability certified and have a globally distributed audience of identifiable users that are highly engaged.

Advertisers from our partner ad networks can identify these apps separately and serve premium, high-yield ads exclusively to these apps for the highest return on investment for their campaigns. For developers, this means that apps that are part of this program can earn more advertising revenue.

How does Microsoft add apps to this program?

We continually review apps in the Store against the [qualifying criteria](#). Apps that meet the qualifying criteria are put through a rigorous manual review process to check for a smooth, well-defined user experience and to make sure ads are well placed to avoid unintentional clicks. Apps that clear both the automatic and manual screening process are then added to the program.

This is currently an invite-only program, and merely meeting the qualifying criteria does not automatically entitle an app to become part of the premium program. If you want to know more about improving your app experience to qualify for the program, please reach out to us at aiacare@microsoft.com.

What are the criteria for apps in the program?

Apps in this program must satisfy the following criteria:

- Minimum guaranteed daily ad request volume.
- High viewability rate for impressions.
- Good quality clicks contributing to a minimum click-through-rate across all ad formats.
- Follow Interactive Advertising Bureau (IAB) standard ad sizes and placement methods. No hidden ads.
- Clean placement of banner ads (for example, ads are not adjacent to each other).
- Ad units are not used across multiple apps.
- [Clear defined user flow](#)  to the point where the video and/or interstitial banner ad is shown. Video ads are not pre-cached.
- Controlled ad refresh behavior so that the ads returned from the mediation server are always played.
- Great Store presence with clear app descriptions, high fidelity screenshots and icons uploaded to the Store. Video trailers are strongly recommended.
- No fraudulent activity.

Can apps get removed from the program?

Apps in the program are monitored on a frequent basis to ensure they continue to meet the qualifying criteria. To sustain a minimum daily ad request volume, make sure your app usage does not drop significantly over time. A great way to do this is by [running regular promotion campaigns](#).

Can I provide additional marketing material to showcase my app in the program?

Any content uploaded as part of the Store submission is automatically used to showcase premium apps. In addition, if you would like to use more app or game screenshots, game trailers and rich descriptions, please reach out to us at aiacare@microsoft.com.

Engage customers with the Microsoft Store Services SDK

Article • 01/04/2023

The Microsoft Store Services SDK provides features that help you engage with customers in your Universal Windows Platform (UWP) apps, such as sending targeted notifications to your apps and running A/B experiments in your apps. This SDK is an extension for Visual Studio 2015 and later versions of Visual Studio.

ⓘ Note

To display ads in your UWP apps, use the [Microsoft Advertising SDK](#) instead of the Microsoft Store Services SDK. The advertising libraries have been moved from the Microsoft Store Services SDK to the Microsoft Advertising SDK. For more information, see [Display ads in your app](#).

Scenarios supported by the Microsoft Store Services SDK

The Microsoft Store Services SDK currently supports the following scenarios for UWP apps. For API reference documentation, see [Microsoft Store Services SDK API reference](#).

Scenario	Description
Run experiments in your UWP app with A/B testing	Run A/B tests in your Universal Windows Platform (UWP) app to measure the effectiveness of features on some customers before you release the features to everyone. After you define an experiment in Partner Center, use the StoreServicesExperimentVariation class to get variations for your experiment in your app, use this data to modify the behavior of the feature you are testing, and then use the LogForVariation method to send view event and conversion events to Partner Center. Finally, use Partner Center to view the results and manage the experiment.
Launch Feedback Hub from your UWP app	Use the StoreServicesFeedbackLauncher class in your UWP app to direct your Windows 10 and Windows 11 customers to Feedback Hub, where they can submit problems, suggestions, and upvotes. Then, manage this feedback in the Feedback report in Partner Center.

Scenario	Description
Configure your UWP app to receive Partner Center push notifications	Use the StoreServicesEngagementManager class in your UWP app to register your app to receive targeted push notifications that you send to your customers using Partner Center.
Log custom events in your UWP app for the Usage report in Partner Center	Use the StoreServicesCustomEventLogger class in your UWP app to log custom events that are associated with your app in Partner Center. Then, review the total occurrences for your custom events in the Custom events section of the Usage report in Partner Center.

Prerequisites

The Microsoft Store Services SDK requires:

- Visual Studio 2015 or a later version.
- Visual Studio Tools for Universal Windows Apps installed with your version of Visual Studio.

Install the SDK

There are two options for installing the Microsoft Store Services SDK on your development computer:

- **MSI installer** You can install the SDK via the MSI installer available [here](#).
- **NuGet package** You can install the SDK as a NuGet package.

Microsoft periodically releases new versions of the Microsoft Store Services SDK with performance improvements and new features. If you have existing projects that use the SDK and you want to use the latest version, download and install the latest version of the SDK on your development computer.

Install via MSI

To install the Microsoft Store Services SDK via the MSI installer:

1. Close all instances of Visual Studio.
2. If you previously installed the Microsoft Store Engagement and Monetization SDK, Universal Ad Client SDK, or Ad Mediator extension, uninstall these SDKs now. Optionally, open a **Command Prompt** window and run these commands to clean out any older SDK versions that may have been installed with Visual Studio, but which may not appear in the list of installed programs on your computer:

Console

```
MsiExec.exe /x{5C87A4DB-31C7-465E-9356-71B485B69EC8}  
MsiExec.exe /x{6AB13C21-C3EC-46E1-8009-6FD5EBEE515B}  
MsiExec.exe /x{6AC81125-8485-463D-9352-3F35A2508C11}
```

3. Download and install the [Microsoft Store Services SDK](#). It may take a few minutes to install. Be sure and wait until the process has finished.
4. Restart Visual Studio.
5. If you have an existing project that references libraries from any earlier version of the Microsoft Store Services SDK, Microsoft Advertising SDK, Universal Ad Client SDK, or Microsoft Store Engagement and Monetization SDK, we recommend that you open your project in Visual Studio and clean and rebuild your project (in **Solution Explorer**, right-click your project node and choose **Clean**, and then right-click your project node again and choose **Rebuild**).

Otherwise, if you are using the SDK for the first time in your project, you are now ready to [add the assembly reference to your project](#).

Install via NuGet

To install the Microsoft Store Services SDK libraries via NuGet:

1. Close all instances of Visual Studio.
2. If you previously installed the Microsoft Store Engagement and Monetization SDK, Universal Ad Client SDK, or Ad Mediator extension, uninstall these SDKs now. Optionally, open a **Command Prompt** window and run these commands to clean out any older SDK versions that may have been installed with Visual Studio, but which may not appear in the list of installed programs on your computer:

Console

```
MsiExec.exe /x{5C87A4DB-31C7-465E-9356-71B485B69EC8}  
MsiExec.exe /x{6AB13C21-C3EC-46E1-8009-6FD5EBEE515B}
```



```
MsiExec.exe /x{6AC81125-8485-463D-9352-3F35A2508C11}
```

3. Start Visual Studio and open the project in which you want to use the Microsoft Store Services SDK.

ⓘ **Note**

If your project already includes library references from an earlier MSI installation of the SDK, remove these references from your project. These references will have warning icons next to them because the libraries they reference were removed in the previous steps.

4. In Visual Studio, click **Project** and **Manage NuGet Packages**.
5. In the search box, type **Microsoft.Services.Store.Engagement** and install the Microsoft.Services.Store.Engagement package. When the package is done installing, save your solution.

ⓘ **Note**

If the **Output** window reports an *Install-Package* error that indicates the specified path is too long, you may need to configure NuGet to extract packages to an alternate location with a shorter path than the default location. To do this, add the `repositoryPath` value to a `nuget.config` file on your computer and assign it to a short folder path where NuGet packages can be extracted. For more information, see [this article](#) in the NuGet documentation. Alternatively, you can try moving your Visual Studio project to an alternate folder with a shorter path. The problem could also be caused by your global packages path being too long. In this case, add the `globalPackagesFolder` value into your `nuget.config` file.

6. Close the Visual Studio solution that contains your project and then reopen the solution.
7. If your project already references libraries from an earlier version of the Microsoft Store Services SDK that was installed via NuGet and you have updated your project to a newer release of the SDK, we recommend that you clean and rebuild your project (in **Solution Explorer**, right-click your project node and choose **Clean**, and then right-click your project node again and choose **Rebuild**).

Otherwise, if you are using the SDK for the first time in your project, you are now ready to [add the assembly reference to your project](#).

Add the assembly reference to your project

After you install the Microsoft Store Services SDK via the MSI installer or NuGet, follow these instructions to reference the SDK assembly in your UWP project.

1. Open your project in Visual Studio.

ⓘ Note

If your project is a JavaScript app that targets **Any CPU**, update your project to use an architecture-specific build output (for example, **x86**).

2. In **Solution Explorer**, right click **References** and select **Add Reference...**
3. In **Reference Manager**, expand **Universal Windows**, click **Extensions**, and then select the check box next to **Microsoft Engagement Framework**. This enables you to use the APIs in the [Microsoft.Services.Store.Engagement](#) namespace.
4. Click **OK**.

ⓘ Note

If you installed the SDK libraries via NuGet, your project will contain a **Microsoft.Services.Store.Engagement** reference. The **Microsoft.Services.Store.Engagement** reference represents the NuGet package (rather than the libraries in it), and you can ignore it.

Understanding framework packages in the SDK

The **Microsoft.Services.Store.Engagement.dll** library in the Microsoft Store Services SDK is configured as a *framework package*. This library contains the APIs in the [Microsoft.Services.Store.Engagement](#) namespace.

Because this library is a framework package, this means that after a user installs a version of your app that uses this library, this library is automatically updated on their device through Windows Update whenever we publish a new version of the library with fixes and performance improvements. This helps to ensure that your customers always have the latest available version of the library installed on their devices.

If we release a new version of the SDK that introduces new APIs or features in this library, you will need to install the latest version of the SDK to use those features. In this scenario, you would also need to publish your updated app to the Store.

Related topics

- [Microsoft Store Services SDK API reference](#)
- [Run experiments with A/B testing](#)
- [Launch Feedback Hub from your app](#)
- [Configure your app to receive Partner Center push notifications](#)
- [Log custom events for Partner Center](#)

Run app experiments with A/B testing

Article • 01/04/2023

You can use Partner Center to define remote variables that you can retrieve at run time from your Universal Windows Platform (UWP) apps, and you can test variations of these values with your users to identify the most effective values for driving desired user behavior. Your app can use remote variables to configure app experiences such as in-app purchases, sign-up flow, captions, and ad placements.

The goal of your A/B test should be to identify a variation of your remote variable values that is likely to earn you improved conversion rates (for example, more in-app purchases) by providing a more engaging app experience. After you have identified a successful variation, you can immediately end the experiment and enable that variation for your entire user audience from Partner Center, without having to republish your app.

Create and run an A/B test

To create and run an A/B test, follow these steps:

1. [Create a project and define remote variables in Partner Center](#). This project contains the variables and default variable values for your experiments.
2. [Code your app for experimentation](#). Use an API in the Microsoft Store Services SDK to get remote variable values from the project you created in Partner Center, use this data to modify the behavior of the feature you are testing, and send view event and conversion events to Partner Center.
3. [Define your experiment in Partner Center](#). Create an experiment in your project that defines the unique goals and variations for your A/B test.
4. [Run and manage your experiment in Partner Center dashboard](#). Activate your experiment, and use Partner Center to review the results of the experiment and complete the experiment.

For a walkthrough that demonstrates the end-to-end process, see [Create and run your first experiment with A/B testing](#).

Requirements

A/B testing in Partner Center is supported only for UWP apps.

Before you can run experiments with A/B testing, you must set up your development computer:

- Follow the instructions [here](#) to set up your development computer for UWP development.
- [Install the Microsoft Store Services SDK](#). In addition to the API for experiments, this SDK also provides APIs for other features such as displaying ads and directing your customers to Feedback Hub to collect feedback on your app.

Best practices

For the most useful results, we recommend that you follow these recommendations when running experiments with A/B testing:

- Consider running experiments with only two variations with a randomized 50/50 split distribution for variation assignments.
- Run experiments for at least 2 – 4 weeks to gather sufficient data that is statistically significant and actionable.

Related terms

[Expand table](#)

Term	Definition
Project	A collection of remote variables with default values that your app can access by using the Microsoft Store Services SDK. A project can also optionally contain one or more experiments that share the same remote variables.
Experiment	A set of parameters that define an A/B test that your users will receive. Experiments are defined in the scope of a project, and each experiment consists of: • A <i>view event</i> that indicates when the user starts viewing a variation that is part of your experiment. • One or more goals with <i>conversion events</i> that indicate when an objective has been reached. • One or more <i>variations</i> that define the variable data used by your experiment. The <i>control</i> variation uses the default variable values that are defined in the project for the experiment. In addition to the control variation, experiments typically have at least one additional variation with variable values that are unique to the experiment.
Project ID	A unique ID that associates your app with a project in your Partner Center account. You must use this ID to connect with the A/B testing service in your app code to receive variation data and report view and conversion events to Partner Center. For more information, see Code your app for experimentation .

Term	Definition
	Each project, and all experiments in the project, are associated with exactly one project ID. You can use project IDs to help differentiate between different sets of experiments. For example, you might have one set of experiments that you release to testers in your organization and another set of experiments that you release only to external users of your app. An app can reference multiple project IDs if it implements multiple experiments.
Variation	A collection of one or more variables that you are testing in your experiment. Every experiment must have at least one variable and two variations (including the control). An experiment can have up to five variations.
Variable	A value that your app uses to initialize a property or some other value in your app. During an experiment, the value of the variable changes from variation to variation. After you end an experiment, the variable is assigned the value from the variation that you choose release to all users of your app. Variables can have the following types: string, Boolean, double, and integer.
View event	An arbitrary string that represents an activity when the user starts viewing a variation that is part of your experiment. Typically, this is the name of an event in your code. Your app code will send this view event string to Partner Center when the user starts viewing a variation. For more information, see Code your app for experimentation .
Conversion event	An arbitrary string that represents an objective for a goal of an experiment. Typically, this is the name of an event in your code. Your app code will send this conversion event string to Partner Center when the user reaches an objective. For more information, see Code your app for experimentation .

Related topics

- [Create a project and define remote variables in Partner Center](#)
- [Code your app for experimentation](#)
- [Define your experiment in Partner Center](#)
- [Manage your experiment in Partner Center](#)
- [Create and run your first experiment with A/B testing](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Create an experiment project in Partner Center

Article • 10/20/2022

To get started with experimentation, create an experimentation [project](#) for your app in Partner Center and define the remote variables that your app can access.

The following instructions describe the core steps to create a project. For a detailed walkthrough that demonstrates the end-to-end process of creating a project and then running an experiment, see [Create and run your first experiment with A/B testing](#).

Instructions

1. Sign in to [Partner Center](#).
2. Under **Your apps**, select the app for which you want to create an experiment.
3. In the navigation pane, select **Services** and then select **Experimentation**.
4. On the **Experimentation** page, click the **New project** button in the **Projects** section. If you have already created one or more projects, those projects are listed in the **Projects** section.
5. In the **New project** page, enter a name for your new project.
6. In the **Remote variables** section, add the [variables](#) that you want to be available to all experiments in this project, and define default values for each variable. The default values you specify here are used for the control group of the experiments, and for any users who do not participate in the experiment.
7. If the **Remote variables** section is collapsed, click **Show** on the section heading.
8. Click **Add variable** to create each new variable that you want to be available to any experiment in this project, and type the variable name and the default value of the variable.
9. When you are done adding variables, click **Save**.
10. In the **SDK integration** section, make note of the [Project ID](#) value. When you [code your app for experimentation](#), you must reference this project ID in your code so you can receive variation data and report view and conversion events to Partner Center.

ⓘ Note

You cannot edit, add, or remove remote variables while an experiment in the project is active. This limitation helps protect the integrity of the data for the control group for the active experiment.

Next steps

After you create a project, you can [code your app for experimentation](#) to start retrieving remote variable values in your app, and you can [create an experiment in the project](#).

Related topics

- [Code your app for experimentation](#)
- [Define your experiment in Partner Center](#)
- [Manage your experiment in Partner Center](#)
- [Create and run your first experiment with A/B testing](#)
- [Run app experiments with A/B testing](#)

Code your app for experimentation

Article • 10/20/2022

After you [create a project and define remote variables in Partner Center](#), you are ready to update the code in your Universal Windows Platform (UWP) app to:

- Receive remote variable values from Partner Center.
- Use remote variables to configure app experiences for your users.
- Log events to Partner Center that indicate when users have viewed your experiment and performed a desired action (also called a *conversion*).

To add this behavior to your app, you'll use APIs provided by the Microsoft Store Services SDK.

The following sections describe the general process of getting variations for your experiment and logging events to Partner Center. After you code your app for experimentation, you can [define an experiment in Partner Center](#). For a walkthrough that demonstrates the end-to-end process of creating and running an experiment, see [Create and run your first experiment with A/B testing](#).

Note

Some of the experimentation APIs in the Microsoft Store Services SDK use the **asynchronous pattern** to retrieve data from Partner Center. This means that part of the execution of these methods may take place after the methods are invoked, so your app's UI can remain responsive while the operations complete. The asynchronous pattern requires your app to use the **async** keyword and **await** operator when calling the APIs, as demonstrated by the code examples in this article. By convention, asynchronous methods end with **Async**.

Configure your project

To get started, install the Microsoft Store Services SDK on your development computer and add the necessary references to your project.

1. [Install the Microsoft Store Services SDK](#).
2. Open your project in Visual Studio.
3. In Solution Explorer, expand your project node, right-click **References**, and click **Add Reference**.
4. In **Reference Manager**, expand **Universal Windows** and click **Extensions**.

5. In the list of SDKs, select the check box next to **Microsoft Engagement Framework** and click **OK**.

ⓘ Note

The code examples in this article assume that your code file has **using** statements for the **System.Threading.Tasks** and **Microsoft.Services.Store.Engagement** namespaces.

Get variation data and log the view event for your experiment

In your project, locate the code for the feature that you want to modify in your experiment. Add code that retrieves data for a variation, use this data to modify the behavior of the feature you are testing, and then log the view event for your experiment to the A/B testing service in Partner Center.

The specific code you need will depend on your app, but the following example demonstrates the basic process. For a complete code example, see [Create and run your first experiment with A/B testing](#).

C#

```
private StoreServicesExperimentVariation variation;
private StoreServicesCustomEventLogger logger;

// Assign this variable to the project ID for your experiment from Dev
// Center.
// The project ID shown below is for example purposes only.
private string projectId = "F48AC670-4472-4387-AB7D-D65B095153FB";

private async Task InitializeExperiment()
{
    // Get the current cached variation assignment for the experiment.
    var result = await
StoreServicesExperimentVariation.GetCachedVariationAsync(projectId);
    variation = result.ExperimentVariation;

    // Refresh the cached variation assignment if necessary.
    if (result.ErrorCode != StoreServicesEngagementErrorCode.None ||
result.ExperimentVariation.IsStale)
    {
        result = await
StoreServicesExperimentVariation.GetRefreshedVariationAsync(projectId);

        if (result.ErrorCode == StoreServicesEngagementErrorCode.None)
```

```

        {
            variation = result.ExperimentVariation;
        }
    }

    // Get the remote variable named "buttonText" and assign the value
    // to the button.
    var buttonText = variation.GetString("buttonText", "Grey Button");
    await button.Dispatcher.RunAsync(
        Windows.UI.Core.CoreDispatcherPriority.Normal,
        () =>
        {
            button.Content = buttonText;
        });

    // Log the view event named "userViewedButton" to Dev Center.
    if (logger == null)
    {
        logger = StoreServicesCustomEventLogger.GetDefault();
    }

    logger.LogForVariation(variation, "userViewedButton");
}

```

The following steps describe the important parts of this process in detail.

1. Declare a [StoreServicesExperimentVariation](#) object that represents the current variation assignment and a [StoreServicesCustomEventLogger](#) object that you will use to log view and conversion events to Partner Center.

C#

```

private StoreServicesExperimentVariation variation;
private StoreServicesCustomEventLogger logger;

```

2. Declare a string variable that is assigned to the [project ID](#) for the experiment you want to retrieve.

ⓘ Note

You obtain a project ID when you **create a project in Partner Center**. The project ID shown below is for example purposes only.

C#

```

private string projectId = "F48AC670-4472-4387-AB7D-D65B095153FB";

```

3. Get the current cached variation assignment for your experiment by calling the static [GetCachedVariationAsync](#) method, and pass the project ID for your experiment to the method. This method returns a [StoreServicesExperimentVariationResult](#) object that provides access to the variation assignment via the [ExperimentVariation](#) property.

C#

```
var result = await
StoreServicesExperimentVariation.GetCachedVariationAsync(projectId);
variation = result.ExperimentVariation;
```

4. Check the [IsStale](#) property to determine whether the cached variation assignment needs to be refreshed with a remote variation assignment from the server. If it does need to be refreshed, call the static [GetRefreshedVariationAsync](#) method to check for an updated variation assignment from the server and refresh the local cached variation.

C#

```
if (result.ErrorCode != StoreServicesEngagementErrorCode.None ||
result.ExperimentVariation.IsStale)
{
    result = await
StoreServicesExperimentVariation.GetRefreshedVariationAsync(projectId);

    if (result.ErrorCode == StoreServicesEngagementErrorCode.None)
    {
        variation = result.ExperimentVariation;
    }
}
```

5. Use the [GetBoolean](#), [GetDouble](#), [GetInt32](#), or [GetString](#) methods of the [StoreServicesExperimentVariation](#) object to get the values for the variation assignment. In each method, the first parameter is the name of the variation that you want to retrieve (this is the same name of a variation that you enter in Partner Center). The second parameter is the default value that the method should return if it is not able to retrieve the specified value from Partner Center (for example, if there is no network connectivity), and a cached version of the variation is not available.

The following example uses [GetString](#) to get a variable named *buttonText* and specifies a default variable value of **Grey Button**.

C#

```
var buttonText = variation.GetString("buttonText", "Grey Button");
```

6. In your code, use the variable values to modify the behavior of the feature you are testing. For example, the following code assigns the *buttonText* value to the content of a button in your app. This example assumes you have already defined this button elsewhere in your project.

C#

```
await button.Dispatcher.RunAsync(  
    Windows.UI.Core.CoreDispatcherPriority.Normal,  
    () =>  
    {  
        button.Content = buttonText;  
    });
```

7. Finally, log the [view event](#) for your experiment to the A/B testing service in Partner Center. Initialize the `logger` field to a [StoreServicesCustomEventLogger](#) object and call the [LogForVariation](#) method. Pass the [StoreServicesExperimentVariation](#) object that represents the current variation assignment (this object provides context about the event to Partner Center) and the name of the view event for your experiment. This must match the view event name that you enter for your experiment in Partner Center. Your code should log the view event when the user starts viewing a variation that is part of your experiment.

The following example shows how to log a view event named **userViewedButton**. In this example, the goal of the experiment is to get the user to click a button in the app, so the view event is logged after the app has retrieved the variation data (in this case, the button text) and assigned it to the content of the button.

C#

```
if (logger == null)  
{  
    logger = StoreServicesCustomEventLogger.Default;  
}  
  
logger.LogForVariation(variation, "userViewedButton");
```

Log conversion events to Partner Center

Next, add code that logs [conversion events](#) to the A/B testing service in Partner Center. Your code should log a conversion event when the user reaches an objective for your